Brian Hodge 5/13/26

The xv6 lottery scheduler was implemented by changing the kernel so the CPU is shared between processes based on how many tickets each process has. I added tickets and ticks variables to the process structure in kernel/proc.h to store a process's priority and track how much CPU time it has used. In the scheduler() function inside kernel/proc.c, the scheduler adds up all the tickets from runnable processes, randomly picks a winning ticket, and gives the CPU to the process that owns that ticket. I also created the settickets() system call so ticket amounts can be changed and the getpinfo() system call to view process information using a custom pstat structure. Finally, it was tested with the lotterytest and ps_lottery programs that showed processes with more tickets received more CPU time.

```
init: starting sh
$ lotterytest
$ ps_lottery

PID     TICKETS TICKS
7       1       6
4       30      349
5       20      236
6       10      125
$ $
```

```
  GNU nano 8.4                          kernel/syscall.h
// System call numbers
#define SYS_fork    1
#define SYS_exit    2
#define SYS_wait    3
#define SYS_pipe    4
#define SYS_read    5
#define SYS_kill    6
#define SYS_exec    7
#define SYS_fstat   8
#define SYS_chdir   9
#define SYS_dup    10
#define SYS_getpid 11
#define SYS_sbrk   12
#define SYS_pause  13
#define SYS_uptime 14
#define SYS_open   15
#define SYS_write  16
#define SYS_mknod  17
#define SYS_unlink 18
#define SYS_link   19
#define SYS_mkdir  20
#define SYS_close  21
#define SYS_getprocs 22
#define SYS_sem_wait   23
#define SYS_sem_signal 24
#define SYS_sleep   25
#define SYS_settickets 26
#define SYS_getpinfo   27
```

```
  GNU nano 8.4                                    user/usys.pl
print "# generated by usys.pl - do not edit\n";

print "#include \"kernel/syscall.h\"\n";

sub entry {
    my $prefix = "sys_";
    my $name = shift;
    if ($name eq "sbrk") {
        print ".global $prefix$name\n";
        print "$prefix$name:\n";
    } else {
        print ".global $name\n";
        print "$name:\n";
    }
    print " li a7, SYS_${name}\n";
    print " ecall\n";
    print " ret\n";
}

entry("fork");
entry("exit");
entry("sem_wait");
entry("sem_signal");
entry("sleep");
entry("getprocs");
entry("wait");
entry("pipe");
entry("read");
entry("write");
entry("close");
entry("kill");
entry("exec");
entry("open");
entry("mknod");
entry("unlink");
entry("fstat");
entry("link");
entry("mkdir");
entry("chdir");
entry("dup");
entry("getpid");
entry("sbrk");
entry("pause");
entry("uptime");
entry("settickets");
entry("getpinfo");
```

```
  GNU nano 8.4                                  kernel/sysproc.c
    }
    sleep(&ticks, &tickslock);
  }

  release(&tickslock);
  return 0;
}

uint64
sys_settickets(void)
{
  int n;
  argint(0, &n); // Call it separately
  if(n < 1)      // Just check the value of n
    return -1;

  acquire(&myproc()->lock);
  myproc()->tickets = n;
  release(&myproc()->lock);
  return 0;
}


uint64
sys_getpinfo(void)
{
  struct pstat st;
  struct proc *p;
  uint64 addr;

  argaddr(0, &addr); // Call it separately

  int i = 0;
  for(p = proc; p < &proc[NPROC]; p++){
    acquire(&p->lock);
    st.inuse[i] = (p->state != UNUSED);
    st.pid[i] = p->pid;
    st.tickets[i] = p->tickets;
    st.ticks[i] = p->ticks;
    release(&p->lock);
    i++;
  }

  return copyout(myproc()->pagetable, addr, (char *)&st, sizeof(st));
}
```

```
  GNU nano 8.4                                  kernel/pstat.h
#ifndef _PSTAT_H_
#define _PSTAT_H_
#include "param.h"
struct pstat {
  int inuse[NPROC];
  int tickets[NPROC];
  int pid[NPROC];
  int ticks[NPROC];
};
#endif
```

```
  GNU nano 8.4                                  kernel/proc.h

  // p->lock must be held when using these:
  enum procstate state;        // Process state
  void *chan;                  // If non-zero, sleeping on chan
  int killed;                  // If non-zero, have been killed
  int xstate;                  // Exit status to be returned to parent's >
  int pid;                     // Process ID

  int tickets;   // Number of lottery tickets
  int ticks;     // Number of times scheduled
```

GNU nano 8.4 user/user.h

```
#define SBRK_ERROR ((char *)-1)

struct pstat;
struct stat;

// system calls
int fork(void);
int exit(int) __attribute__((noreturn));
int wait(int*);
int settickets(int);
int getpinfo(struct pstat*);
```

GNU nano 8.4 user/lotterytest.c

```
#include "kernel/types.h"
#include "user/user.h"

int main() {
  int counts[] = {30, 20, 10};

  for (int i = 0; i < 3; i++) {
    int pid = fork();
    if (pid == 0) { // Child process
      settickets(counts[i]);
      for (;;) {} // Work loop
    }
  }
  exit(0);
}
```

GNU nano 8.4 user/ps_lottery.c

```
#include "kernel/types.h"
#include "kernel/pstat.h"
#include "user/user.h"

int main() {
  struct pstat st;
  if (getpinfo(&st) < 0) {
    printf("Error\n");
    exit(1);
  }
  printf("PID\tTICKETS\tTICKS\n");
  for (int i = 0; i < 64; i++) {
    if (st.inuse[i] && st.pid[i] > 2) { // Skip init/sh
      printf("%d\t%d\t%d\n", st.pid[i], st.tickets[i], st.ticks[i]);
    }
  }
  exit(0);
}
```

```
UPROGS=\
        $U/_cat\
        $U/_echo\
        $U/_semtest\
        $U/_getprocs\
        $U/_qlocktest\
        $U/_lotterytest\
        $U/_ps_lottery\
```

```
  GNU nano 8.4                                  kernel/proc.c
//    via swtch back to the scheduler.
void
scheduler(void)
{
  struct proc *p;
  struct cpu *c = mycpu();
  c->proc = 0;
  for(;;){
    intr_on();

    int total_tickets = 0;
    for(p = proc; p < &proc[NPROC]; p++){
      acquire(&p->lock);
      if(p->state == RUNNABLE){
        total_tickets += p->tickets;
      }
      release(&p->lock);
    }

    if(total_tickets > 0){
      // Simple pseudo-random number logic
      static unsigned int seed = 123;
      seed = seed * 1103515245 + 12345;
      int golden_ticket = seed % total_tickets;

      int ticket_cnt = 0;
      for(p = proc; p < &proc[NPROC]; p++){
        acquire(&p->lock);
        if(p->state == RUNNABLE){
          ticket_cnt += p->tickets;
          if(ticket_cnt > golden_ticket){
            p->state = RUNNING;
            p->ticks++;
            printf("TICK_LOG: %d\n", p->pid);
            c->proc = p;
            swtch(&c->context, &p->context);
            c->proc = 0;
            release(&p->lock);
            break;
          }
        }
        release(&p->lock);
      }
    }
  }
}
```